

Тема: Модульне тестування програмного коду (Unit Testing)

Мета: Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

Вихідний код реалізованого модуля з коментарями:

Посилання на ЛР 2:

<https://github.com/Ruslan62/bse-lr1-Usatenko/tree/main/reports/LR2>

Реалізовано на основі діаграми класів:

https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/reports/LR2/class_diagram.png

Реалізований клас з коментарями:

https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/src/main/cipher_configuration.py

Таблиця проєктування тестів:

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус (pass/fail)
test_init_aes_128_success	algorithm="AES", key_size=128	Створено успішно	EP	pass
test_init_unsupported_algo	algorithm="UNKNOWN", key_size=128	Виняток ValueError	EP	pass
test_init_invalid_key_size	algorithm="AES", key_size=100	Виняток ValueError	BVA	pass
test_generate_key_128	seed_phrase="password"	Рядок "PAS7-PAS14"	EP	pass
test_generate	seed_phrase=	Рядок	EP	pass

_key_alt	"alternative"	"ALT7-ALT14"		
test_generate_key_short_seed	seed_phrase="12"	Виняток ValueError	BVA	pass
test_process_batch_success	data_packets=["packet1", "packet2"]	{"processed": 2, "failed": 0}	EP	pass
test_process_batch_empty_list	data_packets=[]	{"processed": 0, "failed": 0}	BVA	pass
test_process_batch_single_item	data_packets=["only_one"]	{"processed": 1, "failed": 0}	BVA	pass
test_process_batch_num	data_packets=["pass123", "crypt56"]	{"processed": 2, "failed": 0}	EP	pass

Вихідний код тестового набору з коментарями:

https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/src/test/test_cipher.py

За допомогою інструменту pytest було згенеровано звіт, який зафіксував загальне покриття рядків коду на позначці 82%.

Скріншот покриття коду з відображенням відсотку line coverage:

```

src\test\test_cipher.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.7-final-0
-----

Name                               Stmts  Miss  Cover
-----
src\main\cipher_configuration.py     49     9    82%
TOTAL                               49     9    82%
Coverage HTML written to dir htmlcov
===== 10 passed in 0.14s =====

```

Доповнення тестового набору для збільшення покриття:

Тест-кейс	Вхідні дані	Очікуваний	Техніка	Статус
-----------	-------------	------------	---------	--------

		результат	(EP/BVA)	(pass/fail)
test_init_rsa_success	algorithm="RSA", key_size=2048	Об'єкт створено успішно	EP	pass
test_process_batch_with_error	data_packets=["valid", 123]	{"processed": 1, "failed": 1}	EP	pass

Після першого запуску 10 базових тестів було отримано покриття коду 82%. Для збільшення покриття було додано 2 нових тести. У результаті вийшло 12 успішних тестів, які покрили раніше не задіяні шматки коду, завдяки чому фінальне покриття піднялося до 92% .

Скріншоти покриття коду:

```
src\test\test_cipher.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.7-final-0

Name                               Stmts  Miss  Cover
-----
src\main\cipher_configuration.py     49      4   92%
TOTAL                                49      4   92%
Coverage HTML written to dir htmlcov
===== 12 passed in 0.17s =====
```

```

1 class CipherConfiguration:
2     """Клас для керування криптографічними конфігураціями та контролю пакетів даних."""
3
4     def __init__(self, algorithm: str, key_size: int, transformation: str = "AES/CBC/PKCS5Padding"):
5         """
6         Ініціалізація конфігурації, перевірка сумісності криптоалгоритмів та встановлення безпечних лімітів довжини ключів.
7         """
8         self.algorithm = algorithm.strip().upper()
9         self.transformation = transformation
10
11         # Визначення масиву дозволених розмірів ключів для кожного підтримуваного стандарту
12         if self.algorithm == "AES":
13             self.allowed_sizes = [128, 192, 256]
14         elif self.algorithm == "RSA":
15             self.allowed_sizes = [2048, 4096]
16         elif self.algorithm == "DES":
17             self.allowed_sizes = [56, 64]
18         else:
19             # Захист системи від запуску несертифікованих або вразливих алгоритмів
20             raise ValueError(f"Алгоритм '{algorithm}' не підтримується. Оберіть AES, RSA або DES.")
21
22         # Контроль відповідності довжини ключа специфікаціям обраного алгоритму
23         if key_size not in self.allowed_sizes:
24             raise ValueError(f"Розмір ключа {key_size} біт не підходить для алгоритму {self.algorithm}.")
25
26         self.key_size = key_size
27         self.history_logs = [] # Журнал для фіксації виконаних системою криптооперацій
28
29     def generate_key(self, seed_phrase: str) -> str:
30         """
31         Генерація шаблону ключа - створення детермінованої текстової послідовності на основі користувацької секретної фрази (seed).
32         """
33         if not seed_phrase or len(seed_phrase.strip()) < 3:
34             raise ValueError("Фраза-зерно (seed) має містити хоча б 3 символи.")
35
36         # Розрахунок кількості структурних блоків ключа залежно від його загальної біткової довжини
37         blocks_count = self.key_size // 64
38         if blocks_count == 0:
39             blocks_count = 1 # Запобігання нульовій довжині для низькобітного стандарту DES
40
41         generated_key = ""
42         cleaned_seed = seed_phrase.strip().upper()
43
44         # Покрокове формування унікальних сегментів ключа за допомогою математичного зміщення фрази
45         for i in range(1, blocks_count + 1):
46             block_part = f"{cleaned_seed[:3]}{i * 7}"
47             generated_key += block_part + "-"
48
49         final_key = generated_key.rstrip("-")
50         self.history_logs.append(f"Згенеровано шаблон ключа: {final_key}")
51         return final_key
52
53     def process_batch_data(self, data_packets: list) -> dict:
54         """
55         Пакетна обробка даних - аналіз та пре-валідація масиву вхідних повідомлень перед їх передачею в конвеєр шифрування.
56         """
57         if not isinstance(data_packets, list):
58             raise TypeError("Вхідні дані повинні бути представлені списком пакетів.")
59
60         report = {"processed": 0, "failed": 0, "details": []}
61
62         # Послідовний перебір та ізольований аналіз кожного пакету з отриманої черги даних
63         while index < len(data_packets):
64             packet = data_packets[index]
65
66             # Контроль цілісності структури та фільтрація пошкодженого вмісту в окремому пакеті
67             try:
68                 if not isinstance(packet, str):
69                     raise TypeError("Вміст пакету має бути виключно текстом.")
70                 if len(packet.strip()) == 0:
71                     raise ValueError("Порожній пакет не підлягає обробці.")
72
73                 # Реєстрація успішного проходження валідації для поточного елемента
74                 report["details"].append(f"Пакет {index} успішно перевірено ({len(packet)} симв.).")
75                 report["processed"] += 1
76
77             except (TypeError, ValueError) as error:
78                 # Перехоплення локальних помилок для запобігання аварійній зупинці всього потоку обробки
79                 report["details"].append(f"Пакет {index} заблоковано. Причина: {str(error)}")
80                 report["failed"] += 1
81
82             index += 1 # Зміщення покажчика на наступний елемент черги даних
83
84         self.history_logs.append(f"Оброблено пакетів: {report['processed']}, помилок: {report['failed']}")
85         return report

```

Бонус: налаштувати автоматичний запуск тестів через GitHub Actions:

файл для автоматичного запуску тестів при кожному push до репозиторію:
<https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/.github/workflows/python-tests.yml>

```
name: Unit Tests
on: [push, pull_request]

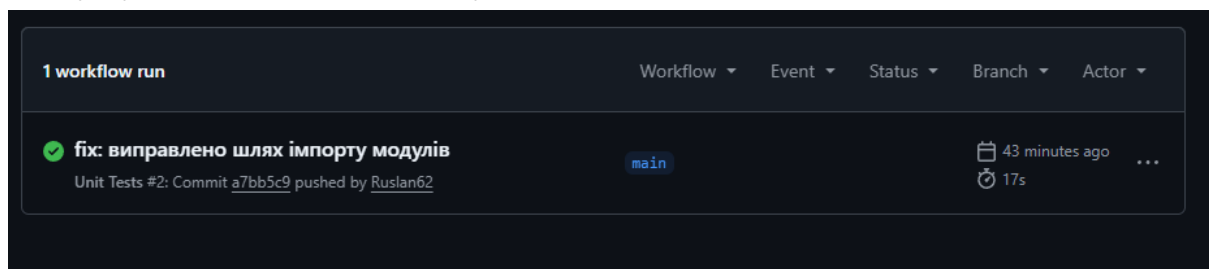
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install pytest pytest-cov

      - name: Execute pytest with coverage
        run: python -m pytest --cov=src.main.cipher_configuration --cov-fail-under=80 src/test/
```

Статус успішного виконання у вкладці GitHub Actions:



звіт утиліти pytest-cov із покриттям коду 92%:

```
Execute pytest with coverage

1 ▶ Run python -m pytest --cov=src.main.cipher_configuration --cov-fail-under=80 src/test/
11 ===== test session starts =====
12 platform linux -- Python 3.12.13, pytest-9.0.3, pluggy-1.6.0
13 rootdir: /home/runner/work/bse-lr1-Usatenko/bse-lr1-Usatenko
14 plugins: cov-7.1.0
15 collected 12 items
16
17 src/test/test_cipher.py ..... [100%]
18
19 ===== tests coverage =====
20 _____ coverage: platform linux, python 3.12.13-final-0 _____
21
22 Name                               Stmts  Miss  Cover
23 -----
24 src/main/cipher_configuration.py    49      4   92%
25 -----
26 TOTAL                               49      4   92%
27 Required test coverage of 80% reached. Total coverage: 91.84%
28 ===== 12 passed in 0.07s =====
```

Висновки: У ході лабораторної роботи було опановано модульне тестування, аналіз покриття та налаштування автоматизації через GitHub Actions. Початковий набір тестів забезпечив покриття коду на 82%, після додавання двох нових кейсів підсумковий показник зріс до 92%. Головною проблемою стало те, що в програмі залишилися непокриті ділянки коду, які тести взагалі не зачепили під час виконання і шляхами поліпшення стали розширення тестового набору новими кейсами для перевірки цих пропущених гілок.